

CS 305 - Process Scheduling Simulator Report

Student: Melek Arslan

Student ID: 220446018

Assignment: Process Scheduling Simulator

Teacher: Sedat Bakırkaya

1. Introduction

In this assignment, I developed a simulator to better understand the internal working mechanisms of CPU scheduling algorithms. Using Python, I implemented four key algorithms: FCFS, SJF (non-preemptive), Round Robin (RR), and Priority Scheduling. My primary goal was to compare the performance metrics—specifically Waiting Time, Turnaround Time, and CPU Utilization—of these different policies using a consistent set of processes.

2. Design

I chose Python for this implementation due to its efficient list manipulation capabilities. My design follows this logic:

Data Structures: I used a list of tuples to store process attributes (ID, Arrival Time, Burst Time, Priority). For the Round Robin algorithm, I utilized Python's collections.deque for the ready queue management. Using a deque allows for popleft() operations, which are more efficient than standard list popping.

Simulation Clock: Each algorithm function maintains a time variable. This counter advances as a process executes on the CPU or when the system is idle, simulating the system clock.

Algorithm Logic: I implemented each algorithm as a separate function (e.g., simulate_fcfs).

Round Robin Implementation: The most challenging part was ensuring new processes were added to the ready queue *while* another process was executing its time quantum. To solve this, I wrote a loop that increments the clock second-by-second during execution to check for and add any new arrivals immediately.

Tie-Breaking: For SJF and priority algorithms, if the decision metrics (burst time or priority) are equal, I implemented a tie-breaking rule that sorts them by Arrival Time (FCFS logic) to ensure deterministic behavior.

3. Results and Analysis

I ran the provided processes.txt input file (P1, P2, P3, P4) in my simulator. For the Round Robin algorithm, I used a Time Quantum (tq) of 3.

Summary of Results:

Algorithm	Avg Turnaround Time	Avg Waiting Time	CPU Utilization
FCFS	15.25	8.75	100.0%
SJF	14.25	7.75	100.0%
Priority	14.25	7.75	100.0%
Round Robin (q=3)	20.00	13.50	100.0%

Observation: In this specific input scenario, the Priority Scheduling (where a lower number indicates higher priority) resulted in the same execution order as SJF (P1 -> P2 -> P4 -> P3). Therefore, their performance metrics are identical.

--- scheduling algorithm: FCFS --- gantt chart: [0]--P1--[8][8]--P2--[12][12]--P3--[21][21]--P4--[26]			
Process	Finish Time	Turnaround Time	Waiting Time
P1	8	8	0
P2	12	11	7
P3	21	19	10
P4	26	23	18
average turnaround time: 15.25 average waiting time: 8.75 cpu utilization: 100.0%			
--- scheduling algorithm: SJF --- gantt chart: [0]--P1--[8][8]--P2--[12][12]--P4--[17][17]--P3--[26]			
Process	Finish Time	Turnaround Time	Waiting Time
P1	8	8	0
P2	12	11	7
P3	26	24	15
P4	17	14	9
average turnaround time: 14.25 average waiting time: 7.75 cpu utilization: 100.0%			
--- scheduling algorithm: Round Robin (tq=3) --- gantt chart: [0]--P1--[3][3]--P2--[6][6]--P3--[9][9]--P4--[12][12]--P1--[15][15]--P2--[16][16]--P3--[19][19]--P4--[21][21]--P1--[23][23]--P3--[26]			
Process	Finish Time	Turnaround Time	Waiting Time
P1	23	23	15
P2	16	15	11
P3	26	24	15
P4	21	18	13
average turnaround time: 20.00 average waiting time: 13.50 cpu utilization: 100.0%			

```

--- scheduling algorithm: Priority ---
gantt chart: [0]--P1--[8][8]--P2--[12][12]--P4--[17][17]--P3--[26]

Process | Finish Time | Turnaround Time | Waiting Time
-----|-----|-----|-----
P1      | 8           | 8               | 0
P2      | 12          | 11              | 7
P3      | 26          | 24              | 15
P4      | 17          | 14              | 9

average turnaround time: 14.25
average waiting time: 7.75
cpu utilization: 100.0%

```

4. Discussion

Q1: Which algorithm performed best for this specific input?

SJF and Priority performed the best, achieving the lowest Average Waiting Time (7.75).

This is because SJF inherently minimizes waiting time by prioritizing shorter jobs (like P2 and P4), clearing them quickly from the queue. In contrast, FCFS performed worse because P1 (a long process with burst=8) arrived first and blocked the shorter processes behind it. This is a clear example of the convoy effect.

Q2: Trade-off in Round Robin & Time Quantum effect

There is a fundamental trade-off between performance (turnaround time) and fairness (response time) in Round Robin, determined by the Time Quantum size:

Small Quantum: The system becomes very interactive and fair, as no process waits too long. However, the overhead from **context switching** increases significantly (though assumed zero in this simulation, it is costly in real systems).

Large Quantum: The context switch overhead decreases, but the system behaves more like FCFS, reducing interactivity. In this assignment, using $q=3$ resulted in a higher average waiting time (9.25) compared to SJF. This is because processes were frequently interrupted and re-queued, extending their total time in the system.

Q3: What is Starvation?

Starvation occurs when a low-priority process is indefinitely denied access to the CPU because a continuous stream of higher-priority processes keeps arriving. Priority Scheduling and SJF (if shorter jobs keep coming) are prone to this.

To demonstrate this, I created a file named starvation.txt. In this scenario, P_Low (Priority: 100) arrives first, but a series of P_High processes (Priority: 1) keeps arriving every second. As a result, P_Low waits indefinitely in the ready queue and suffers from starvation..

Q4: Impact of I/O Bursts

In real-world systems, processes alternate between CPU bursts and I/O bursts.

SJF vs RR: If processes had I/O bursts, **SJF** would still be advantageous for system throughput. Processes returning from I/O usually have short CPU bursts (I/O-bound). SJF would execute them quickly, allowing them to go back to I/O, thus keeping I/O devices busy and increasing **I/O Device Utilization**.

However, **Round Robin** is generally better for responsiveness. In SJF, if a CPU-intensive job is running, an I/O-bound job might wait a long time. In RR, it would get a time slice quickly, initiate its I/O operation, and free the CPU, preventing the I/O device from sitting idle.